

Lab 3 — Advanced SQL: Aggregation, Subqueries, CTEs & Windows

Module I · Week 3 · Duration: 2 hours **Dataset:** data/store.sql · **Deliverable:** lab03_solution.sql + short answers

Objectives

Apply Lectures 7–9 on the larger **Store** dataset (an online retailer):

1. Aggregation with GROUP BY / HAVING.
 2. Scalar, IN/EXISTS, and **correlated** subqueries.
 3. **CTEs** (WITH), including one **recursive** CTE.
-

Setup (≈10 min)

```
sqlite3 lab3.db < data/store.sql
sqlite3 lab3.db ".tables"      # categories, customers, products, orders, order_items
```

Schema and the key derived value:

```
customers(cust_id, name, city, country, join_date)
categories(cat_id, cat_name)
products(prod_id, prod_name, cat_id, price, stock)
orders(order_id, cust_id, order_date, status)
order_items(order_id, prod_id, quantity, unit_price)
```

```
-- revenue of a line item = quantity * unit_price
```

Get oriented:

```
SELECT COUNT(*) FROM orders;      -- ~65
SELECT COUNT(*) FROM order_items;  -- ~159
```

Part 1 — Aggregation & grouping

1. Number of products and average price **per category** (show cat_name).
2. **Total revenue per category**, highest first (categories → products → order_items, sum quantity*unit_price).
3. Total revenue **per customer country**.
4. Customers who placed **more than 3** orders (HAVING COUNT(*) > 3).
5. For **each product**, total units sold — **including products never sold** (show 0). *Hint:* LEFT JOIN + COALESCE(SUM(...),0).
6. Months (YYYY-MM via strftime('%Y-%m', order_date)) whose total revenue exceeded 15,000.
7. Per category: product count, average price, units sold, and revenue in **one** query (the “grouped report” pattern).

□ In Q2/Q7 you join order_items (one-to-many) — make sure you sum quantity*unit_price, not a per-order value, to avoid double counting.

□ **Checkpoint:** the number of categories in Q1 is 6.

Part 2 — Subqueries

8. Products priced **above the overall average** product price (scalar subquery).
9. Products priced **above their own category's average** price (**correlated** subquery on `cat_id`).
10. Customers who have **never placed an order** (NOT EXISTS; explain in a comment why NOT IN would be risky here).
11. Customers who ordered in **2023** but **not** in **2022** (two EXISTS/NOT EXISTS, using `strftime('%Y', order_date)`).
12. The product(s) with the **maximum** price (subquery returning the max).
13. **Division:** customers who have bought **every** product in the 'Books' category (double NOT EXISTS). *If none qualify, explain how you verified that, and identify the customer who comes closest.*

□ **Checkpoint:** Q12 returns the priciest product (Noise-Cancel Headset, 8999.00) unless your generated data differs — verify against `SELECT MAX(price) FROM products;`

Part 3 — CTEs

14. Using a **CTE** `cust_revenue(cust_id, revenue)`, list the **top-spending customer per country** (join the CTE to `customers`; one row per country).
15. **Recursive CTE** (switch to `university.sql` for this one): all **transitive prerequisites** of 'CS-347'.

```
sqlite3 uni.db < ../lab01/data/university.sql    # or datasets/university.sql
```

You **cannot** filter on a window column (rank) in the same query's WHERE. Compute it in a CTE/subquery, then filter in the outer query.

Checkpoint: Q15 returns CS-101 (CS-347 requires CS-101).

Stretch goals (optional)

- Rewrite Q10 as a `LEFT JOIN ... IS NULL` and confirm the same customers.
-

Submission

Submit `lab03_solution.sql` containing all 15 queries (commented by number), the two short written answers (A) as comment blocks.

Self-check: the script runs end-to-end against a freshly loaded `store.sql` with no errors; checkpoint values match; Q15 runs against `university.sql`.